

Formatted Representation of Economic Models: An Econ-ARK Initiative

Computational economic models are often hard-coded. Today, computing methods advance faster than economics research. There is a need to represent models separate from solution techniques. We do that, inspired by Dolang and DYNARE, Our current focus is on heterogenous agent macroeconomics and macrofinance models

1 INTRODUCTION

Econ-ARK is an open source project that aims to improve reproducibility in economics research and applications.

Currently, most economic models are bespoke and hard-coded. There is not a standard way to represent or compute over models. This makes replication and reproducibility more difficult. Economic models are often more computationally complex than other kinds of simulations because they involve rational actors which typically must solve rather complex optimization problems. This makes tying modeling to computational 'solvers' tempting.

We are designing and implemented a new way of representing economic models. This format contains the data needed to define the model, but is agnostic to the software solver. Importantly, this will allow the same model to be solved using different, comparable, approximate methods. This will enable better bench-marking of computational results within economics research and applications.

We are currently implementing a reference implementation of this language as we approach the 1.0 version of our flagship economics library, HARK. You can find out more about this at <https://econ-ark.org/>.

We are inspired by other open source projects in the Economics space, such as DYNARE and Dolo. However, we aspire to enable the representation of a wider range of models than either of these tools allow.

2 BLOCKS

One principle of the modeling format is *modularity*.

We want users to be able to define elements of models, and recombine them into new ones.

2.1 Example: Consumption

For example, here is an example of the markup for part of a consumption-savings problem:

```
name: consumption
shocks:
  theta: !MeanOneLogNormal
  sigma: 0.1
dynamics:
  b: k * R
  m: b + theta
  c: !Control
  info: m
  upper_bound: m
  a: m - c
reward:
  u: c ** (1 - CRRA) / (1 - CRRA)
```

Author's address:

This block corresponds to the following equations:

$$\begin{aligned}
 \theta &\sim \text{Lognormal}(1, 0.1) \\
 b &= Rk \\
 m &= b + \theta \\
 a &= m - c \\
 c &< m
 \end{aligned} \tag{1}$$

$$u(c) = \frac{c^{1-\rho}}{1-\rho}$$

It identifies that c is a decision (or control) variable, and the information set for that control is m . Given a decision rule $c(m)$, this system of equations governs the dynamics from k to a .

Notice:

- This is a flavor of YAML.
- The equations can all be easily parsed by SymPy into expressions, and used programmatically.
- The equations refer to some variables, such as R and $CRRA$, which are not defined in the block.

Values for free parameters may be provided in calibration section of the document.

calibration:

```

DiscFac: 0.96
CRRA: 2.0
R: 1.03
Rfree: 1.03
EqP: 0.02
LivPrb: 0.98
RiskyStd: 0.1

```

Or these free parameters might be set in other blocks.

2.2 Example: Portfolio choice

Consider another block. This block represents the choice of a consumer to allocate savings to either a risky or risk free asset.

```

name: portfolio choice
shocks:
  risky_return: !Lognormal
    mean: Rfree + EqP
    std: 0.2
dynamics:
  alpha: !Control
    info: a
    lower_bound: 0
    upper_bound: 1
  R: Rfree + (risky_return - Rfree) * alpha

```

This corresponds to the following equations:

$$\begin{aligned}
 \sigma_\eta &= \log((0.2)^2 / (R_{free} + E)^2 + 1) \\
 \mu &= \log(R_{free} + E) - \sigma_\eta / 2 \\
 \eta &\sim \text{Logormal}(\mu, \sigma_\eta) \\
 \alpha &\in [0, 1] \\
 R &= R_{free} + (\eta - R_{free})\alpha
 \end{aligned} \tag{2}$$

In this block, R , the rate of return, is set endogenously.

3 SEQUENCING

Blocks can be composed into sequences in order to create multi-stage dynamic programming problems.

The two blocks above compose into a single problem, which can be characterized by the following Bellman equation:

$$\begin{aligned}
 v(k, R) &= \mathbb{E}_\theta \left(\max_c u(c) + \beta \mathbb{E}_\eta [\max_{a'} v(a, R')] \right) \\
 \theta &\sim \text{Lognormal}(1, 0.1) \\
 b &= Rk \\
 m &= b + \theta \\
 a &= m - c \\
 c &< m \\
 u(c) &= \frac{c^{1-\rho}}{1-\rho} \\
 \sigma_\eta &= \log((0.2)^2 / (R_{free} + E)^2 + 1) \\
 \mu &= \log(R_{free} + E) - \sigma_\eta / 2 \\
 \eta &\sim \text{Logormal}(\mu, \sigma_\eta) \\
 \alpha &\in [0, 1] \\
 R' &= R_{free} + (\eta - R_{free})\alpha
 \end{aligned} \tag{3}$$